CS 5370 Group Project
Fall 2026

Building and Investigating a Small Transformer Language Model

1. Project Overview

Your group will design, implement, train, and evaluate a small language model (SLM) using the decoder-only Transformer architecture studied in class. The project simulates the major stages of language-model development on a scale appropriate for your available computing resources. Each group will create its own fictional dataset using generative AI tools or agents, train a model from scratch, and investigate a research question related to its learning or behavior.

2. Fictional Domain and Dataset Generation

Invent a fictional organization or world, such as a company, airline, university, or technology firm. Establish a consistent knowledge base describing its entities, facts, policies, relationships, and activities.

However, limit your project to one narrowly defined department, service, or activity within that organization, rather than attempting to model the entire organization. Otherwise, you risk creating a dataset containing too many distinct facts, entities, and relationships for your small model to learn effectively within the available time and computational resources. This could lead to poor learning, unreliable responses, and unnecessarily long training times.

The goal is not to maximize the amount of information in your dataset, but to help a small model learn a limited set of facts through repeated exposure to varied linguistic expressions.

For example, if you invent an airline, you might focus exclusively on its lost-baggage department, including its reporting procedures, compensation policies, and customer service operations.

Use generative AI tools or agents to create a corpus of realistic text based on this knowledge base. Include different document types and writing styles, such as policies, internal communications, reports, and customer notices.

Emphasize linguistic diversity rather than the number of facts. The same information should appear repeatedly, but expressed in different ways and in different contexts.

For example, a seven-day baggage reporting requirement might appear in a formal company policy, an employee email, and a customer notice, each using different wording.

Your generated documents must remain consistent with your established knowledge base. Avoid continuously introducing new facts, entities, or policies simply to increase the size of the dataset.

Keep the corpus manageable for your available computing resources. Report its size in tokens after tokenization.

Prepare training, validation, and test partitions, ensuring that evaluation material is appropriately separated from training data.

3. Tokenization and Vocabulary

Choose a tokenization strategy appropriate for your corpus and computational resources.

Construct the vocabulary, assign token IDs, and implement text encoding and decoding.

Explain how vocabulary size determines the dimensions of the embedding matrix and the output size of the language-model head.

4. Design and Implement the Transformer

Implement a decoder-only Transformer, drawing directly from concepts seen in class. Your architecture must include:

Token embeddings and positional information to represent input tokens and their positions.

Causal multi-head self-attention, including learned projection matrices (W_Q), (W_K), (W_V), and (W_O).

Causal masking to prevent tokens from accessing future positions.

Decoder blocks containing attention, feed-forward neural networks, residual connections, and normalization.

An LM head producing one logit for each vocabulary token.

Select the number of decoder blocks, attention heads, embedding dimensions, and context length according to your hardware capabilities.

You may use PyTorch for tensor operations, automatic differentiation, and optimization. However, you must construct and understand the Transformer architecture rather than simply load a pretrained language model.

5. Pretraining: Next-Token Prediction

Train your model from randomly initialized weights on the fictional text corpus.

The objective is to predict the next token from the preceding tokens, using cross-entropy loss.

Use backpropagation and an appropriate optimizer to update the trainable parameters, including embeddings, attention projections, feed-forward weights, and LM head weights.

Monitor training and validation loss and save model checkpoints.

At this stage, the objective is to learn the statistical and linguistic patterns of the fictional domain; not specifically to behave as a question-answering assistant.

6. Text Generation and Evaluation

Use your pretrained model to generate text autoregressively from prompts.

Evaluate its behavior using held-out examples and appropriate quantitative measures, such as validation loss.

Examine whether the model can generate plausible domain-specific text, complete familiar statements, and reproduce relationships from the training corpus.

Document successful and unsuccessful outputs.

7. Supervised Fine-Tuning for Question Answering

After pretraining, your group must perform a short supervised fine-tuning stage using question–answer examples derived from the same fictional knowledge base.

Continue training your pretrained Transformer on these examples to teach it to generate answers to questions about your fictional organization.

Evaluate and compare the model's question-answering performance before and after fine-tuning. Reserve a separate set of evaluation questions, including previously unseen question formulations, that must not be used during fine-tuning. Use the same evaluation questions for both the pretrained and fine-tuned models to allow a fair comparison of their performance.

Discuss what improved, what did not, and the limitations of your fine-tuned model.

8. Research Investigation

Formulate one research question and investigate it using evidence from your model.

Example questions include (you don't necessarily have to use these):

What linguistic and factual patterns can a small Transformer learn from a synthetic corpus?

How does generated text change across training checkpoints?

How reliably can the model reproduce factual relationships from its training data?

How does sampling temperature affect generated text?

How does supervised fine-tuning affect the model's ability to answer questions?

You are not required to train multiple models. Your investigation may use saved checkpoints, held-out examples, and controlled generation settings.

Discuss your findings, limitations, and possible explanations.

9. Final Deliverables

Each group must submit the following:

a) **Research report (5–7 pages):** Dataset, research question, model architecture, training procedure, evaluation, findings, and limitations.
b) **Synthetic dataset:** Generated corpus, fictional knowledge base, and data-generation methodology.
c) **Source code:** Data processing, model implementation, training, evaluation, and text generation.
d) **Trained models:** Submit two model checkpoints: one saved after pretraining and another saved after supervised fine-tuning. Include the tokenizer/vocabulary and model configuration needed to load and run both versions. These checkpoints will allow us to compare the model's behavior before and after fine-tuning.
e) Results: Training and validation loss curves, training logs, generated text samples, and experimental findings.
f) **Demonstration video (3–5 minutes):** A screen recording showing your trained model being loaded and used to generate text from several prompts. At least one prompt should be different from those used in your training examples. The demonstration must show the actual program running, not merely screenshots or prepared outputs.

g) **README and execution instructions:** Provide clear instructions enabling us to load your trained model and generate text from new prompts. Include installation requirements, hardware requirements, commands for running inference, required files, and example inputs. We should be able to run your trained model without retraining it.

h) **In-class presentation and Q&A:** Each group will deliver a slide presentation explaining its fictional dataset, Transformer architecture, training process, research question, experimental results, and conclusions. The presentation will be followed by a Q&A session in which group members must demonstrate their understanding of the project, including the technical concepts covered in class. All four group members are expected to participate and be prepared to answer questions about any aspect of the project, not just the component they personally worked on.

i) **AI Use Statement:** Include a short AI Use Statement describing which AI tools you used and the main ways you used them. Substantial assistance with code, experimental design, analysis, or writing must be disclosed. You are responsible for verifying any code, analysis, or claims produced with AI assistance.

j) **Reproducibility requirement:** Your submitted checkpoint, code, tokenizer, and configuration must be compatible and sufficient to run inference. We may test your model using prompts not included in your demonstration. A nonfunctional submission cannot be substituted with screenshots, generated text, or a demonstration video alone.

k) **Member Contributions:** Include a brief statement describing each group member's contributions to the project.